Search ... ⌘ K

» GO2 Steamdeck

GO2 Steamdeck

Quick Start

This guide provides instructions for interfacing with the **Unitree GO2 Edu** robot using **Ubuntu 20.04** and **ROS2 Foxy**.



Note

- Ensure that you have already completed the setup and pairing with your GO2 Edu.
- Verify all functionality using the mobile app before continuing.
- Initial setup instructions can be found in the **GO2 Docs**.
- This guide builds upon the information available in those docs.

> **Warning**
>
> The documentation below is valid **only for the GO2 Edu version** (tested until 08-July-2025).

> **Warning**
>
> Unitree is constantly updating their firmware, and at times changing naming conventions may cause breakage in some newer firmwares. If this occurs, either **submit an issue** or **create a pull request** with the correct feature updates.

Static Network Connection

For the **first-time connection**, you must configure the robot's network via a LAN cable.

1. On your **PC** (not on the robot), open:

   Settings → Network → Add (+) New Connection

2. In the **IPv4** settings:

   - Change the method to **Manual**
   - Set the **IP Address**: `192.168.123.51`
   - Set the **Netmask**: `24`
   - Save and restart your network connection

3. Confirm that the host's local IP is configured correctly:

   ```
   ifconfig
   ```

4. Ping the robot to verify the connection:

   ```
   ping 192.168.123.18
   ```

5. Access the robot via **SSH**:

```
ssh -X unitree@192.168.123.18
```

Default password:

```
123
```

IP Addresses

IP Address	Device	Username	Password
192.168.123.161	GO2 MCU	x	x
192.168.123.18	GO2 Auxiliary PC	unitree	123
192.168.123.18:9000	GO2 Webserver MBS	x	mybotshop

> **Warning**
>
> Other active networks may interfere with your connection. For best results, disconnect from all other networks and connect **only** to the GO2.

Network Connection

Connection via LAN:

```
ssh -X unitree@192.168.123.18
```

When starting up, keep in mind:

- The **Go2 wrapper** operates on `ROS_DOMAIN_ID=10` . Launch files automatically use **Domain ID 10**.
- The **namespace** can be changed to match your robot's ID for multi-robot systems.

Drivers Startup

- After completing installation, the ROS2 packages will run in the background (if pre-installed by the **mybotshop** team).

Important

- To check which packages must run, see the `go2_bringup` startup script.
- Service names are mentioned there and can be referenced directly.

Example:

```
sudo service go2-webserver status
ros2 launch go2_webserver webserver.launch.py
```

- Ensure duplicate services are not running simultaneously. Close unnecessary services before launching new ones.



Gaits

Available gait modes:

Normal Trot (recommended for Nav2, SLAM applications):

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

Flat Terrain Mode (for flat environments):

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

Economic Trot (long runs, battery conservation):

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

Fast Trot:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

A.I Trot:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

Sports Mode

> **Important**
>
> The **Sports Mode** feature is **deprecated** in **GO2 v2.2.1+** or with the latest firmware update.

Switch to sports mode:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/modes go2_interface/
```

Visualization

Launch **RViz** visualization:

```
ros2 launch go2_viz view_robot.launch.py
```

GO2 Tele-operation

Run teleoperation with keyboard:

```
ROS_DOMAIN_ID=10 ros2 run teleop_twist_keyboard teleop_twist_keyboard -
```

ROS2 Modules

This section describes the **ROS2 modules** available for the Unitree GO2 platform, their usage, and example commands.

Webserver

The **GO2 Webserver** module comes pre-installed for heavy integration projects.

It can be accessed directly via:

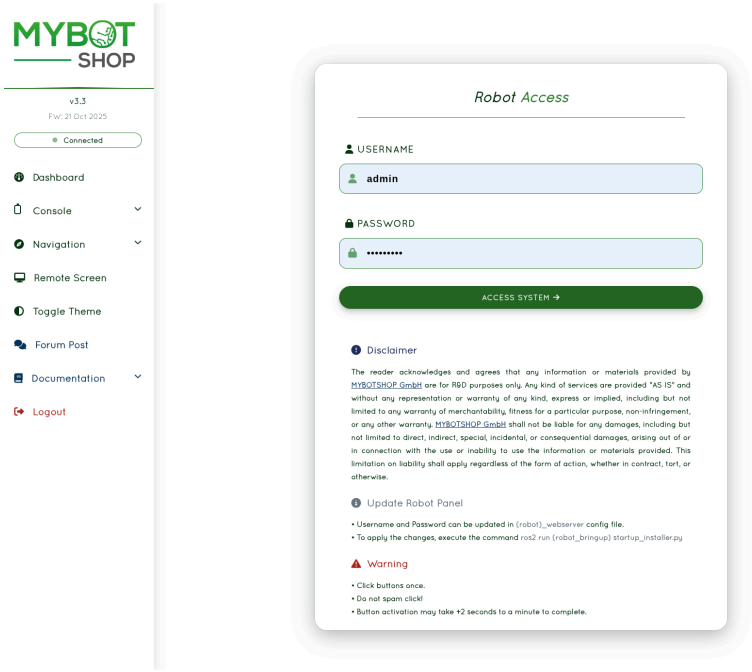
- **LAN**: http://192.168.123.18:9000/
- **Wi-Fi**: The IP assigned to the robot

The configuration file is located at:

```
/opt/mybotshop/src/mybotshop/go2_webserver/config/robot_webserver.yaml
```

Login

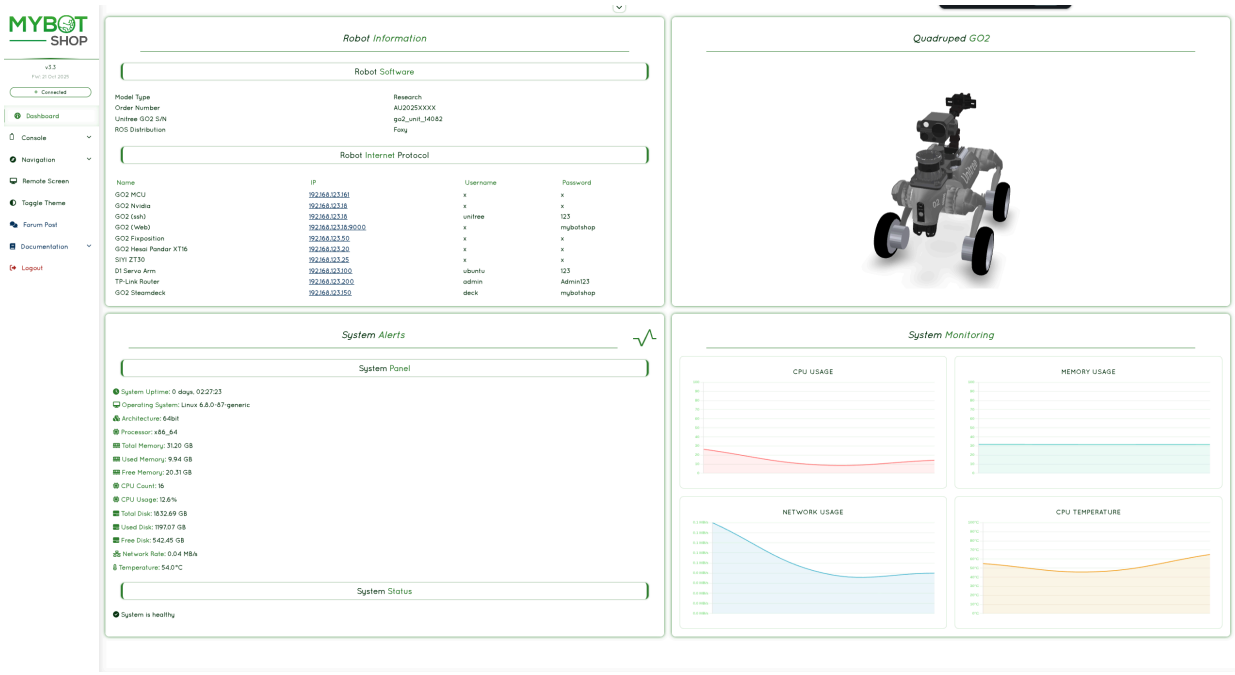
The login page allows authorized access to the GO2 webserver interface.



Dashboard

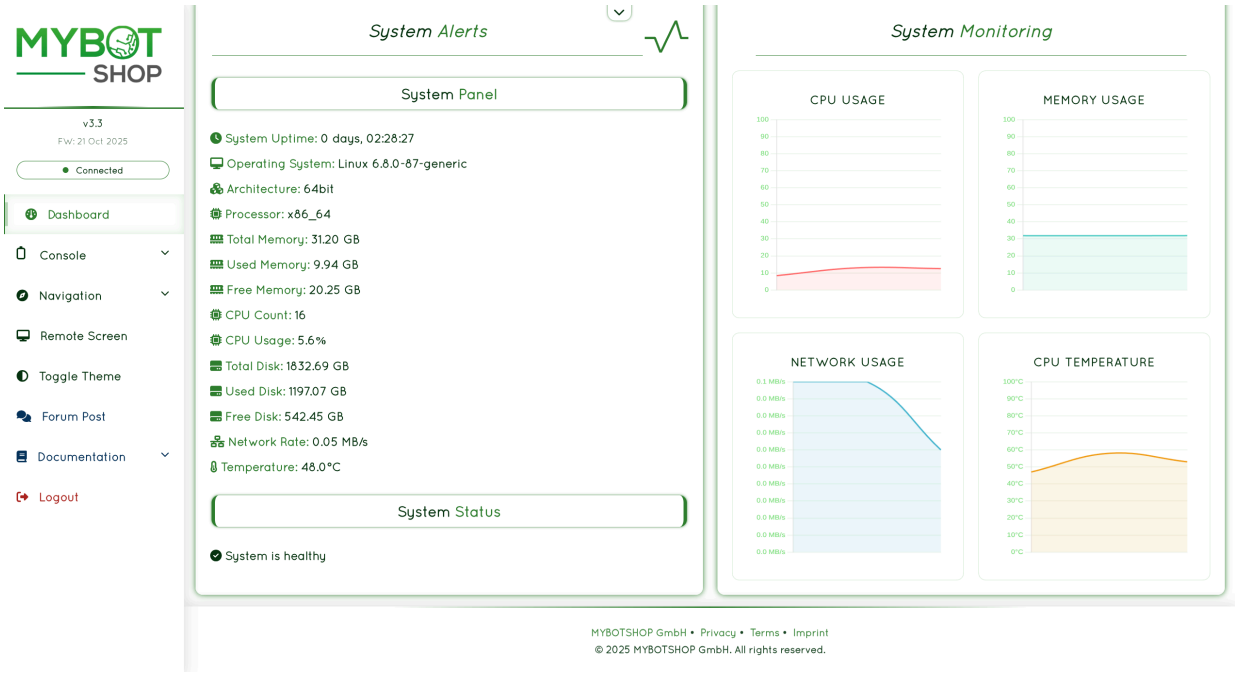
The **Dashboard** provides key system controls:

- Enable or disable the GO2 ROS2 services.
- Record system logs for debugging or analysis.



System

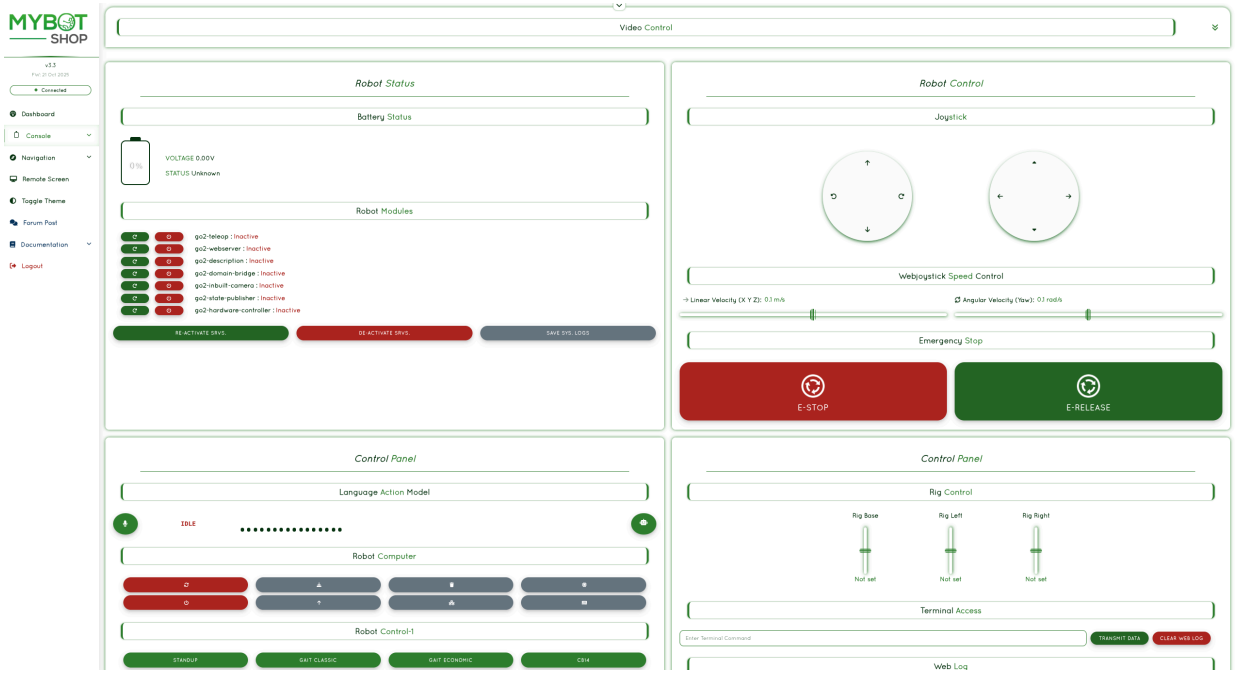
The **System** page displays the external PC status of the Unitree GO2. - Battery status is only active when ROS2 drivers are enabled from the Dashboard.



Console

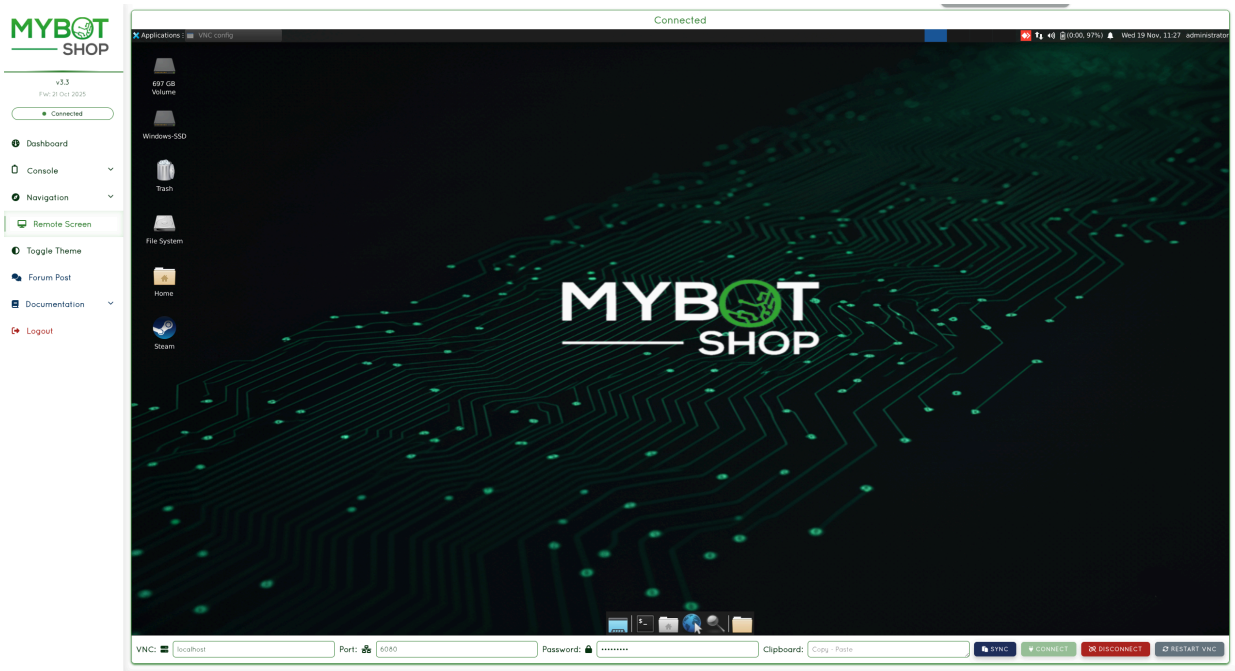
The **Console** allows direct control over the Unitree GO2:

- Move the robot and switch between different gaits.
- Adaptable to new ROS2 services.
- Record ROS2 bags for later playback.
- Access the GO2 external speaker if available.



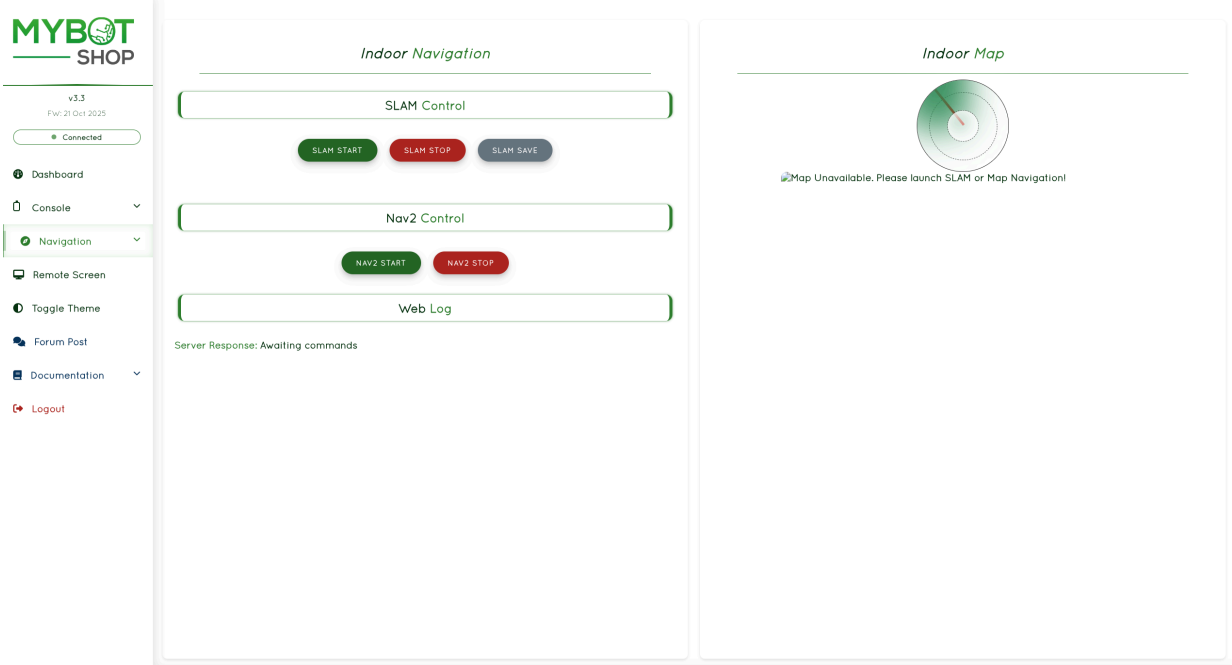
Remote Desktop

Access the on-board screen of the GO2's computer directly via the webserver.



Navi Indoor

This feature is **coming soon**.



Navi Outdoor

This feature is **coming soon**.

LLM Interface

This feature is **coming soon**.

Mode Activation

The following **modes** are available via ROS2 services:

- balance_stand
- recovery_stand
- stand_down
- stand_up
- stretch
- damp
- sit

- rise_sit
- front_jump
- front_pounce
- front_flip
- stop_move

> **Note**
>
> These functions can be extended in the `go2_platform.cpp` file to support new firmware features from the Unitree team.

Available functionalities via ROS2 services:

- **Light level:** 0–10
- **Volume level:** 0–10

Example — **Activating stand-up mode**:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_001/modes go2_interface/sr
```

Wait a few seconds, then switch to balance stand:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_001/modes go2_interface/sr
```

Example — **Standing down**:

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_001/modes go2_interface/sr
```

> **Note**

First use `stand_up` to remove balance mode, then `stand_down`. This is required for the latest firmware and GO2-W.

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_001/modes go2_interface/sr
```

Example — **Changing light level (0–10):**

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/light go2_interface/
```

Example — **Changing volume level (0–10):**

```
ROS_DOMAIN_ID=10 ros2 service call /go2_unit_21576/volume go2_interface
```

Note

Sometimes modes (e.g., `stand`) may fail to activate. In such cases, use the **recovery stand** mode.

Realsense Depth Cameras

Warning

If running **both Realsense cameras**, launch them in **separate terminals** with a **10s delay** to avoid conflicts.

Realsense D405:

```
ros2 launch go2_depth_camera realsense_d405.launch.py
```

Realsense D435i:

```
ros2 launch go2_depth_camera realsense_d435i.launch.py
```

Note

- By default, both **Realsense D435i** and **D405** are disabled.
- The launch file is preconfigured to stream depth data continuously from D435i without lag. Modify parameters in:
  `go2_depth_camera/launch/realsense_d4XX.launch.py`



Lidar Modules

Livox Mid360:

```
ros2 launch go2_lidars livox_mid360.launch.py
```

Hesai Pandar XT-16:

```
ros2 launch go2_lidars hesai.launch.py
```

After launching, access the sensor's web UI and configure:

- **Spin rate**: 300
- **Return mode**: strongest return
- **Azimuth**: 30 → 330

Fixposition Setup

Install dependencies:

```
sudo apt update
sudo apt install libyaml-cpp-dev libboost-all-dev zlib1g-dev libeigen3-
```

Build Fixposition:

```
distrobox enter ros2_humble
rm /opt/mybotshop/src/third_party/gps/07Aug2025_fixposition_humble/COLC
cp -r /opt/mybotshop/src/third_party/gps/07Aug2025_fixposition_humble /
cd /home/unitree/fixposition_humble
source /opt/ros/humble/setup.bash
./setup_ros_ws.sh -r 2
./create_ros_ws.sh /home/unitree/fixposition_humble_ws
cd /home/unitree/fixposition_humble_ws
colcon build --symlink-install --cmake-args -DBUILD_TESTING=OFF && sour
```

Run Fixposition:

```
distrobox enter ros2_humble
source /opt/ros/humble/setup.bash
source /home/unitree/fixposition_humble_ws/install/setup.bash
ros2 launch fixposition_driver_ros2 fixposition_humble.launch.py
```

Run converter (new terminal):

```
ros2 launch odom_to_tf_ros2 odom_to_tf.launch.py
```

Open Manipulator Control

Start the custom ROS driver:

```
ros2 launch go2_manipulation openmanipulator.launch.py
```

> **Note**
>
> This driver must always be running when using **MoveIt** or joint trajectories.

Launch MoveIt & State Publisher:

```
ros2 launch go2_manipulation moveit2.launch.py
```

Open Manipulator Commands

> **Note**
>
> Update the namespace accordingly if required.

Test driver:

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_001/open_manipulator/j
  trajectory: {
    joint_names: [joint1, joint2, joint3, joint4],
    points: [
      { positions: [0.1, 0.1, 0.1, 0.1], time_from_start: { sec: 2 } },
      { positions: [-0.1, -0.1, -0.1, -0.1], time_from_start: { sec: 4
      { positions: [0, 0, 0, 0], time_from_start: { sec: 6 } }
    ]
  }
}"
```

Home position (before turning off, GO2 prior to 06 Mar 2025):

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_001/open_manipulator/j
```

```
  trajectory: {
    joint_names: [joint1, joint2, joint3, joint4],
    points: [
      { positions: [0.0, -1.4, 1.2, 1.2], time_from_start: { sec: 2 } }
    ]
  }
}"
```

Stand position:

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_21576/open_manipulator
  trajectory: {
    joint_names: [joint1, joint2, joint3, joint4],
    points: [
      { positions: [0.0, -0.2, -0.1, -0.24], time_from_start: { sec: 1
    ]
  }
}"
```

Home position (GO2 after 06 Mar 2025):

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_21576/open_manipulator
  trajectory: {
    joint_names: [joint1, joint2, joint3, joint4],
    points: [
      { positions: [0.0, -1.7, 1.4, 1.2], time_from_start: { sec: 2 } }
    ]
  }
}"
```

Close gripper (before turning off):

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_21576/open_manipulator
  trajectory: {
    joint_names: [gripper],
    points: [
      { positions: [1.0], time_from_start: { sec: 2 } },
    ]
  }
}"
```

Open gripper:

```
ROS_DOMAIN_ID=10 ros2 action send_goal /go2_unit_21576/open_manipulator
  trajectory: {
    joint_names: [gripper],
    points: [
      { positions: [-1.0], time_from_start: { sec: 2 } },
    ]
  }
}"
```

Navigation

This section covers **SLAM**, **navigation**, **startup services**, and **simulation tools** for the Unitree GO2 robot.

SLAM (Simultaneous Localization and Mapping)

> **Warning**
>
> The SLAM feature is **not yet ported** to the new system.

1. Ensure `go2_bringup` is running.
2. Launch SLAM:

```
ros2 launch go2_navigation slam.launch.py
```

3. Begin mapping using teleoperation at **0.2 m/s** (keyboard control recommended).
4. Once satisfied with your map, export it:

```
ROS_DOMAIN_ID=10 ros2 run nav2_map_server map_saver_cli -f /opt/mybo
```

5. Rebuild so that custom maps can be located (required if the map name is **not `map_`**):

```
cd /opt/mybotshop/ && colcon build --symlink-install && source /opt/
```

Odometric Navigation

> Note
>
> The **Nav2 Goal Tool** and **Nav2 Goal Plugin** do not work due to namespace issues. Instead, use **2D Goal Pose** in RViz or custom scripts.

Launch odometric navigation:

```
ros2 launch go2_navigation odom_navi.launch.py
```

Map Navigation

> Note
>
> The **Nav2 Goal Tool** and **Nav2 Goal Plugin** do not work due to namespace issues. Use **2D Goal Pose** or scripts instead.

1. Ensure a map is generated and available in the ROS package. (After saving, place the map in the correct location and rebuild with `colcon build`.)

2. Launch map-based navigation:

```
ros2 launch go2_navigation map_navi.launch.py
```

Startup Services

By default, the GO2 does **not** include a startup job unless explicitly configured. It only launches:

`go2_bringup/system.launch.py`

To verify if a startup service exists:

```
sudo service go2—<ros2 module name> status
```

- **Error message**: `Unit <ros2 module name>.service could not be found.` → No startup service is installed.
- **Markers**:

Color	Meaning
Red	Startup job has **failed**
Green	Service is **running correctly**
Grey	Service has **not yet started**

Restart a service if red/grey:

```
sudo service go2—<ros2 module name> restart
```

To modify or extend startup jobs, edit:

`go2_bringup/startup_installer.py`

Then update:

```
ros2 run go2_bringup startup_installer.py
```

GO2 Steamdeck

Follow the initialization guide:

Steamdeck Initialization Docs ↗

1. Connect to the **custom router** of the GO2 (startup may take several minutes).

Default IP addresses:

Device	IP Address
Steamdeck	192.168.123.150
Router	192.168.123.100
Unitree GO2	192.168.123.18

2. Once the joystick opens, hold **L1** and use the **left/right joysticks** to move the robot.

3. Change gaits via the **web browser console** (opens automatically at `192.168.123.18:9000`).

4. By default, **ROS2 drivers are off**. Use *Restart All* to enable services for Steamdeck + ROS2 control.

5. Additional services can be added via:

```
/opt/mybotshop/src/mybotshop/go2_webserver/go2_webserver/libroscustom.py
```





GO2 Camera

Camera Calibration

> **Note**
>
> Calibration is already provided. Requires a **7x6 checkerboard**.

Run calibration:

```
ROS_DOMAIN_ID=10 ros2 run camera_calibration cameracalibrator --size 8x
```

GO2 Isaac Sim

> **Note**
>
> Instructions for usage are located in the **humble branch**: `qre-go2-humble` .

GO2 Gazebo Classic (Under Development)

Launch Gazebo simulation:

```
ros2 launch go2_gazebo_classic simulation.launch.py
```

Example — controlling GO2 in Gazebo:

```
ros2 run go2_platform go2_gazebo_demo
```

GO2 Gazebo Citadel

> **Important**
>
> **Deprecated**

Debugging & Miscellaneous

This section provides instructions for **debugging, network setup, camera streaming, manipulator control, and recommended hardware enhancements**.

Debugging

RQT GUI

Launch the RQT GUI for ROS2:

```
ROS_DOMAIN_ID=10 ros2 run rqt_gui rqt_gui --ros-args --remap tf:=/go2_u
```

TF Visualization

View TF tree in RQT:

```
ROS_DOMAIN_ID=10 ros2 run rqt_tf_tree rqt_tf_tree --force-discover --ro
```

Generate TF frames diagram:

```
ROS_DOMAIN_ID=10 ros2 run tf2_tools view_frames.py --force-discover --r
```

Miscellaneous

Internet via LAN

Connect the GO2 to a router with internet access via LAN:

```
sudo ip link set eth0 down && sudo ip link set eth0 up
sudo dhclient eth0
```

Wait 10–30 seconds, then update packages:

```
sudo apt-get update
```

Update date and time automatically:

```
sudo date -s "$(wget --method=HEAD -qSO- --max-redirect=0 google.com 2>
```

Fix GO2 USB 2.0 → 3.0 Port (2024 Models)

1. Backup the current device tree:

```
sudo cp /boot/dtb/kernel_tegra234-p3767-0000-p3768-0000-a0.dtb kernel_t
```

2. Download the latest device tree:

Latest Equipment Tree

3. Copy the updated device tree to the boot directory:

```
sudo cp kernel_tegra234-p3767-0000-p3768-0000-a0.dtb /boot/dtb/ -r
```

4. Restart the system for changes to take effect.

Sync Host and GO2 Computers

Synchronize source files using *rsync*:

```
rsync -avP -t --delete -e ssh src unitree@192.168.123.18://opt/mybotsho
```

Camera Stream via Terminal (Non-ROS)

View video stream directly from terminal (replace `multicast-iface` with the network interface, e.g., `eth0`):

```
gst-launch-1.0 udpsrc address=230.1.1.1 port=1720 multicast-iface=eth0
```

Faster version:

```
gst-launch-1.0 udpsrc address=230.1.1.1 port=1720 multicast-iface=eth0
    application/x-rtp, media=video, encoding-name=H264 ! \
    rtph264depay ! queue max-size-buffers=1 ! h264parse ! queue ! avdec
    videoconvert ! autovideosink
```

Open Manipulator X (Independent)

Launch Open Manipulator X Controller:

```
ros2 launch open_manipulator_x_controller open_manipulator_x_controller
```

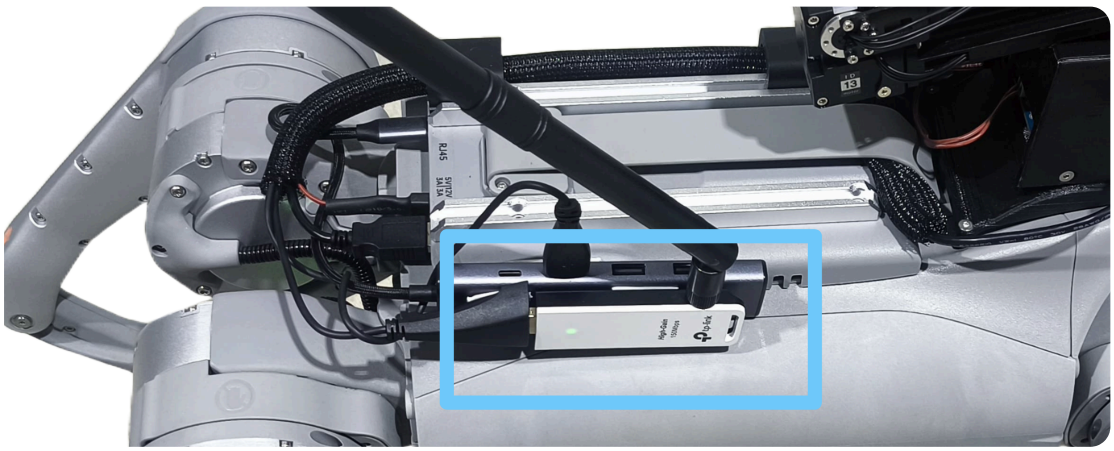
Teleoperate Open Manipulator X:

```
ros2 run open_manipulator_x_teleop teleop_keyboard
```

Enhancements

WiFi Stick

We recommend using a **long-range WiFi stick**, such as **TL-WN722N**, or any high-end long-range WiFi stick for remote operation.



Connection Hub

For connecting a screen or expanding ports, we recommend a **USB-C hub** such as **Ugreen USB-C Hub with HDMI** for testing and debugging.

Latest QRE GO2 Updates

Important

The latest updates are available from the **qre_go2 GitHub repository**. If access is required, email **support@mybotshop.de** with your GitHub username and purchase ID.

Onboard PC Installation

GO2 Open Manipulator

© 2026, QUADRUPED ROBOTICS uG